

Three-Qubit Bit-Flip Error Correction: A Monte Carlo Study of Logical Error Suppression

Abdi Gebisa Nagasa · Ambo University Special Secondary School, Ambo, Ethiopia

Contact: abdigeisa035@gmail.com

Portfolio: <https://abdi-gebisa.github.io/abdi-gebisa-nagasa.github.io/>

Abstract. Quantum information is fragile, and error correction is the central requirement for useful quantum computers. This project tests whether the simplest quantum error-correcting code, the three-qubit repetition code, reduces the effect of bit-flip noise. One logical qubit is encoded across three physical qubits, and 100,000 Monte Carlo trials are run at each of eight physical error probabilities. The simulated logical error rate is compared with an unencoded qubit and with the analytic result $P_L = 3p^2 - 2p^3$, with uncertainty given by binomial standard errors. Logical errors are suppressed at every tested p , by about 97% at $p = 0.01$ and by about 28% at $p = 0.30$, and simulation agrees with theory within statistical error. All results are simulated; no physical quantum processor was used.

1. Research question and hypothesis

Question. How does a three-qubit bit-flip code change the logical error probability as the physical bit-flip rate increases?

Hypothesis. For sufficiently small, independent physical bit-flip probability p , majority-vote correction across three physical qubits yields a logical error probability below p .

2. Objectives

- Build an unencoded single-qubit reference model.
- Encode one logical qubit with the three-qubit repetition code and apply controlled bit-flip noise.
- Decode by majority vote and measure the logical error rate.
- Compare simulation with theory and quantify statistical uncertainty.
- Identify the limits of the code.

3. Theory

A qubit is written (see Nielsen and Chuang [1]) $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$. A bit flip is the Pauli-X operation, $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$. The code maps $|0_L\rangle = |000\rangle$ and $|1_L\rangle = |111\rangle$, so

$$|\psi_L\rangle = \alpha|000\rangle + \beta|111\rangle$$

This is an entangled encoding, not cloning: the no-cloning theorem [2] forbids copying an unknown state into $|\psi\rangle \otimes |\psi\rangle \otimes |\psi\rangle$. Encoding uses two CNOT gates, with the data qubit as control and the other two qubits as targets. Two parity (syndrome) measurements locate a single flipped qubit without revealing α or β , and the flip is then undone.

With independent flips of probability p , the code fails only when two or three qubits flip:

$$P_L = 3p^2(1 - p) + p^3 = 3p^2 - 2p^3$$

An unencoded qubit fails with probability p . The code therefore helps whenever $3p^2 - 2p^3 < p$, which holds for all $p < 0.5$. At small p , $P_L \approx 3p^2$, so errors fall quadratically.

4. Method

A Monte Carlo simulation with a fixed random seed was run for $p \in \{0.01, 0.02, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$, using 100,000 independent trials per value. In the unencoded model one qubit flips with probability p . In the encoded model, each of three qubits flips independently with probability p , and a logical error is recorded when at least two flip. Standard error is $SE = \sqrt{(P_{\text{obs}}(1 - P_{\text{obs}})/N)}$, and the 95% confidence interval is $P_{\text{obs}} \pm 1.96 SE$, where P_{obs} is the observed error fraction.

5. Results

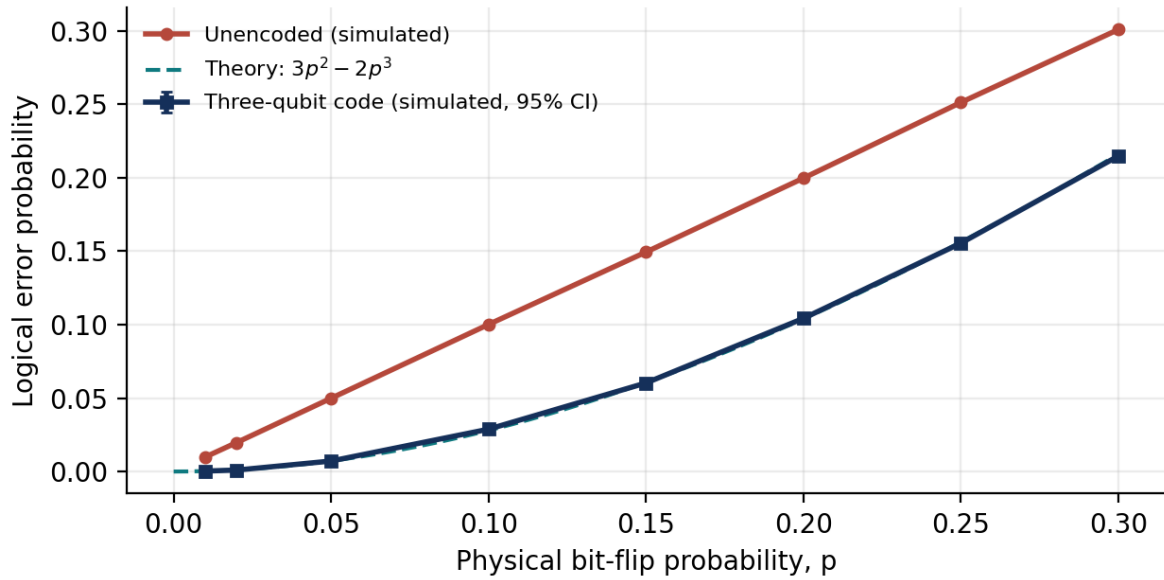


Figure 1. Simulated logical error rate against physical error rate. Error bars ($\pm 1.96 SE$) are smaller than the markers.

Table 1. Simulated error rates ($N = 100,000$ per row).

p	Unencoded	SE	Encoded	SE	Theory	Reduction
0.01	0.00987	0.00031	0.00032	0.00006	0.00030	97%
0.02	0.01973	0.00044	0.00099	0.00010	0.00118	94%
0.05	0.04990	0.00069	0.00715	0.00027	0.00725	86%
0.10	0.10014	0.00095	0.02883	0.00053	0.02800	72%
0.15	0.14944	0.00113	0.06031	0.00075	0.06075	60%
0.20	0.19978	0.00126	0.10429	0.00097	0.10400	48%
0.25	0.25131	0.00137	0.15552	0.00115	0.15625	38%
0.30	0.30094	0.00145	0.21483	0.00130	0.21600	28%

Reduction = $1 - P_{\text{L}}/p$ using the theoretical value. Every encoded measurement lies within about 2 standard errors of theory (largest deviations: $p = 0.02$ and $p = 0.10$).

6. Interpretation

The unencoded rate tracks p , as expected. The encoded rate tracks $3p^2 - 2p^3$ within Monte Carlo fluctuation, confirming that the simulation implements the intended noise model. The hypothesis is supported: logical errors fall below p at every tested value.

The benefit shrinks as noise grows, from a 97% reduction at $p = 0.01$ to 28% at $p = 0.30$, because double and triple flips, which the code cannot fix, become more common. In this idealized model the break-even point is $p = 0.5$; real devices have a much lower threshold, because gates and syndrome measurements are themselves noisy.

7. Limitations

- Only independent bit-flip noise is modeled; phase errors and general errors are not corrected.
- Syndrome extraction and correction are assumed perfect. The simulation applies majority voting directly and does not simulate the syndrome circuit.
- No hardware effects (gate errors, crosstalk, decoherence) are included.
- The repetition code is an educational demonstration, not a fault-tolerant architecture.

8. Connection to modern quantum computing

IBM Quantum documents the repetition code as a basic error-correction example [9, 10]. The code is the building block of Shor's nine-qubit code [3] and of stabilizer codes [4, 5], and was demonstrated experimentally on superconducting qubits by Kelly et al. [6]. Google Quantum AI has shown logical-error suppression as surface-code size increases [11], and later below the surface-code threshold [12]; the surface code itself is described in [7], and the broader context in [8]. This study reproduces the same core principle, redundancy plus syndrome-based correction, at the smallest scale.

9. Conclusion and future work

Under independent bit-flip noise, the three-qubit code lowers the logical error rate at all tested physical error probabilities, matching $3p^2 - 2p^3$. The result illustrates both the power and the limits of simple error correction.

Next steps: add phase-flip and depolarizing noise; simulate noisy syndrome measurements; implement the Shor nine-qubit code; compare with surface-code scaling; and run the repetition code on real quantum hardware.

10. Feedback requested

This manuscript is shared for expert review. Comments are especially welcome on: (i) whether the independent bit-flip model and the binomial uncertainty analysis are appropriate; (ii) how best to extend the simulation to noisy syndrome extraction and to phase-flip or depolarizing noise; and (iii) the framing of the results relative to existing treatments of the repetition code.

11. Code availability and reproducibility

The complete simulation script is given in Appendix A. It uses NumPy only, a fixed seed (2026), $N = 100,000$ trials per probability, and prints the same quantities as Table 1 (observed rates, standard errors, theory). Because each run draws pseudo-random numbers, exact values depend on the seed and library version; any seed gives results statistically consistent with $3p^2 - 2p^3$ (rerunning with the supplied seed gives, for example, 0.02757 ± 0.00052 at $p = 0.10$, against theory 0.02800). A copy of the script is also available on request from the contact address above.

References

1. M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th anniversary ed., Cambridge University Press (2010).
2. W. K. Wootters and W. H. Zurek, "A single quantum cannot be cloned," *Nature* 299, 802-803 (1982).
3. P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Phys. Rev. A* 52, R2493 (1995).
4. A. M. Steane, "Error correcting codes in quantum theory," *Phys. Rev. Lett.* 77, 793 (1996).
5. D. Gottesman, "Stabilizer codes and quantum error correction," PhD thesis, Caltech (1997), arXiv:quant-ph/9705052.
6. J. Kelly et al., "State preservation by repetitive error detection in a superconducting quantum circuit," *Nature* 519, 66-69 (2015).

7. A. G. Fowler, M. Mariantoni, J. M. Martinis and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Phys. Rev. A* 86, 032324 (2012).
8. J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum* 2, 79 (2018).
9. IBM Quantum Learning, "Repetition codes," Foundations of Quantum Error Correction course. <https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/correcting-quantum-errors/repetition-codes> (accessed 4 October 2026).
10. IBM Quantum Documentation, "Repetition codes" (tutorial). <https://quantum.cloud.ibm.com/docs/tutorials/repetition-codes> (accessed 4 October 2026).
11. Google Quantum AI, "Suppressing quantum errors by scaling a surface code logical qubit," *Nature* 614, 676–681 (2023).
12. Google Quantum AI and collaborators, "Quantum error correction below the surface code threshold," *Nature* 638, 920–926 (2025).

All numerical results are simulated data, not measurements from a physical quantum processor.

Appendix A. Simulation code (Python, NumPy)

File: qec_simulation.py. Run with: python qec_simulation.py

```
"""
Monte Carlo simulation of the three-qubit bit-flip repetition code.

For each physical bit-flip probability p, N independent trials are run:
- Unencoded: one qubit flips with probability p.
- Encoded: three qubits flip independently with probability p;
    majority-vote decoding fails if at least two qubits flipped.
Outputs error rates, binomial standard errors, 95% CIs and the theory
curve  $P_L = 3p^2 - 2p^3$ . Requires: numpy (matplotlib optional for the plot).
"""
import numpy as np

SEED = 2026
N_TRIALS = 100_000
P_VALUES = [0.01, 0.02, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30]

def standard_error(p_hat, n):
    return np.sqrt(p_hat * (1.0 - p_hat) / n)

def simulate(p, n, rng):
    unencoded_fail = rng.random(n) < p          # single qubit flips
    flips = rng.random((n, 3)) < p              # three independent flips
    encoded_fail = flips.sum(axis=1) >= 2        # majority vote fails
    return unencoded_fail.mean(), encoded_fail.mean()

def main():
    rng = np.random.default_rng(SEED)
    print(f"{'p':>5} {'unenc':>9} {'SE':>8} {'enc':>9} {'SE':>8} {'theory':>9}")
    for p in P_VALUES:
        u, e = simulate(p, N_TRIALS, rng)
        theory = 3 * p**2 - 2 * p**3
        print(f"{p:5.2f} {u:9.5f} {standard_error(u, N_TRIALS):8.5f} "
              f"{e:9.5f} {standard_error(e, N_TRIALS):8.5f} {theory:9.5f}")

if __name__ == "__main__":
    main()
```